SMART LIBRARY SYSTEM

Complete Project Documentation

Backend • Frontend • Database • Firebase Auth • REST API • Deployment

Submitted By

Lokendra Joshi

Roll Number: 150096725062

Component	Technology / Platform
Frontend	Vanila JS + Vite
Backend	Node.js + Express.js
Authentication	Firebase Auth + JWT
Deployment	Render (Unified Server)

Live Render: https://smart-library-hxsr.onrender.com/

GitHub: https://github.com/ShidoStack/smart-library

1. Project Overview

SmartLibrary is a full-stack, web-based College Library Management System designed to streamline book cataloguing, member management, borrowing, reservations, fines, and real-time notifications. Administrators can manage the entire library inventory and generate reports, while members can browse the catalogue, borrow and reserve books, and track their activity through a secure dashboard.

The application uses a Vanilla JS + Vite frontend, Node.js / Express.js REST API, MongoDB Atlas for persistent storage, Firebase Authentication (Google/Email) for identity management, JWT for session tokens, and Socket.io for real-time updates. The unified production server is deployed on Render.

2. Project Objectives

- Provide a secure, role-based web interface for library administrators and members.
- Allow registered members to browse, borrow, and reserve books online.
- Enforce borrowing rules: due dates, available-copy tracking, and reservation queuing.
- Automatically calculate and track overdue fines per day.
- Provide real-time availability updates via Socket.io when books are borrowed or returned.
- Send in-app notifications to members for due reminders, fines, and reservations.
- Generate administrative reports for borrowing trends, popular books, and member activity.
- Authenticate users using Firebase (Email/Password + Google OAuth) and backend JWT.
- Deploy the full-stack application online using Render with MongoDB Atlas.

3. Technology Stack

Layer	Technology / Platform
Runtime	Node.js v18+
Backend Framework	Express.js
Database	MongoDB Atlas + Mongoose ODM
Authentication	Firebase Admin SDK + Firebase Client SDK
Session Tokens	JSON Web Tokens (JWT) + bcryptjs
Frontend	Vanilla JS + Vite 5
Real-Time	Socket.io 4.x
API Docs	Swagger UI Express

Security	Helmet.js, CORS
Version Control	Git + GitHub
Hosting	Render (Unified Web Service)

4. System Architecture

The system follows a layered MVC architecture:

- Client (Browser) → Vanilla JS + Vite bundled SPA (HTML / CSS / JS)
- Firebase Client SDK → Email / Google Auth → Firebase ID Token
- Frontend → POST /api/auth/firebase → Exchange ID Token for SmartLibrary JWT
- Frontend → HTTPS REST API with Bearer JWT → Express Routes
- Express Routes → Middleware (auth, RBAC) → Controllers → Services
- Services → Mongoose Models → MongoDB Atlas
- Socket.io → Real-time events (book availability, notifications)

4.1 Folder Structure

Path	Purpose
backend/src/server.js	App entry: Express + Socket.io + static frontend serving
backend/src/config/	DB connection, Firebase Admin, env loader
backend/src/routes/	Express route definitions (8 route files)
backend/src/controllers/	Request handlers — auth, books, members, borrow, etc.
backend/src/services/	Business logic layer (MongoDB queries & aggregations)
backend/src/models/	Mongoose schemas: User, Book, BorrowRecord, Fine, Reservation, Notification
backend/src/middleware/	Auth guard (requireAuth), RBAC (requireRole), error handler
backend/src/sockets/	Socket.io event handlers
backend/src/scripts/	Seed script, comprehensive E2E test suite
src/ (Vite frontend)	Vanilla JS SPA: index.html, login.html, register.html
src/js/	api.js (fetch wrapper), store.js (state), auth.js, app.js

5. Environment Configuration

The backend reads all secrets from backend/.env (excluded from Git via .gitignore):

Variable	Purpose
PORT	Local dev port (5001); Render supplies its own
MONGODB_URI	MongoDB Atlas connection string (includes credentials)
JWT_SECRET	Secret key used to sign and verify SmartLibrary JWTs
JWT_EXPIRES_IN	JWT expiry duration (e.g. 7d)
FINE_PER_DAY	Fine amount in currency units per overdue day
BORROW_DURATION_DAYS	Default loan period in days
CLIENT_URL	Allowed CORS origin (Vite dev URL or Render URL)
FIREBASE_SERVICE_ACCOUNT	Firebase Admin credentials JSON (Render secret env var)

Secret values are never committed to GitHub or included in this report.

6. Authentication and Authorization

Authentication uses a dual-layer approach: Firebase handles identity (login / OAuth), and the backend issues its own short-lived JWT for session management.

- Registration: user registers via Firebase Email/Password; backend creates a User document in MongoDB.
- Login: Firebase Client SDK authenticates the user and returns a Firebase ID Token.
- Token Exchange: Frontend POSTs the ID Token to /api/auth/firebase; backend verifies it via Firebase Admin SDK and returns a SmartLibrary JWT.
- Protected Requests: frontend attaches the JWT as Authorization: Bearer <token> for all authenticated API calls.
- RBAC: requireRole middleware restricts admin-only routes (book management, member management, reports).

Method	Endpoint	Purpose	Access
POST	/api/auth/register	Register new user	Public

POST	/api/auth/login	Login (email/password)	Public
POST	/api/auth/firebase	Exchange Firebase token → JWT	Public
GET	/api/auth/me	Get current user profile	Authenticated

7. Database Models

Model	Key Fields / Relationships
User	uid (Firebase), name, email, password (hashed), role (admin/member), membershipId, membershipExpiry
Book	title, author, isbn, genre, totalCopies, availableCopies, coverImage, description, publishedYear
BorrowRecord	book (ref Book), member (ref User), borrowDate, dueDate, returnDate, status (active/returned/overdue)
Fine	borrowRecord (ref), member (ref User), amount, paidAt, status (pending/paid)
Reservation	book (ref Book), member (ref User), reservedAt, status (active/fulfilled/cancelled)
Notification	member (ref User), type, title, message, read (bool), createdAt

8. REST API Reference

8.1 Authentication APIs

Method	Endpoint	Purpose
POST	/api/auth/register	Create member account
POST	/api/auth/login	Authenticate and return JWT
POST	/api/auth/firebase	Exchange Firebase ID Token for JWT
GET	/api/auth/me	Return authenticated user profile

8.2 Books APIs

Method	Endpoint	Purpose
GET	/api/books	Get all books (with search, filter, pagination)
GET	/api/books/:id	Get one book by ID
POST	/api/books	Create a new book (admin only)
PUT	/api/books/:id	Update book details (admin only)
DELETE	/api/books/:id	Delete a book (admin only)

8.3 Members APIs

Method	Endpoint	Purpose
GET	/api/members	Get all members (admin only)
GET	/api/members/:id	Get one member profile
POST	/api/members	Create/register a member
PUT	/api/members/:id	Update member details

8.4 Borrow APIs

Method	Endpoint	Purpose
GET	/api/borrow	Get all borrow records
GET	/api/borrow/user/:memberId	Get borrow history for a member
POST	/api/borrow	Borrow a book (decrements availableCopies)
PUT	/api/borrow/:id/return	Return a borrowed book

8.5 Reservations APIs

Method	Endpoint	Purpose
GET	/api/reservations	Get all reservations
POST	/api/reservations	Reserve a book
DELETE	/api/reservations/:id	Cancel a reservation

8.6 Fines APIs

Method	Endpoint	Purpose
GET	/api/fines	Get all fines (admin)

GET	/api/fines/user/:memberId	Get fines for a member
POST	/api/fines/calculate	Calculate fine for a borrow record
PUT	/api/fines/:id/pay	Mark a fine as paid

8.7 Reports APIs

Method	Endpoint	Purpose
GET	/api/reports/borrowing	Borrowing summary report
GET	/api/reports/popular-books	Most borrowed books report
GET	/api/reports/member-activity	Member activity report

8.8 Notifications APIs

Method	Endpoint	Purpose
GET	/api/notifications	Get notifications for logged-in user
POST	/api/notifications	Create a notification
PATCH	/api/notifications/:id/read	Mark one notification as read
PATCH	/api/notifications/read-all	Mark all notifications as read

9. Business Rules

- A book must have at least one available copy before it can be borrowed.
- Borrowing decrements availableCopies; returning increments it back and emits a Socket.io event.
- Default loan period is 14 days (configurable via BORROW_DURATION_DAYS env var).
- Overdue fine accrues at ₹5 per day (configurable via FINE_PER_DAY env var).
- A member cannot borrow the same book twice concurrently.
- Reservation is only allowed if the book is currently unavailable (all copies borrowed).
- Only administrators can create, update, or delete books and manage all members.
- Members can only view and manage their own borrow records, reservations, fines, and notifications.

10. Frontend Documentation

The Vanilla JS + Vite frontend is a multi-page SPA served from the Express backend in production. It communicates with the backend exclusively through the REST API using the authenticated fetch wrapper in src/js/api.js.

10.1 Pages & Features

Page / Module	Features
login.html	Firebase Email/Password login, Firebase Google OAuth login
register.html	New member registration with membership ID input
index.html	Protected dashboard: book catalogue, borrow, reserve, return, fines, reports, notifications, admin panel
src/js/api.js	Fetch wrapper — auto-injects JWT Bearer token, handles errors
src/js/store.js	In-memory state management — books, members, session, notifications
src/js/auth.js	Firebase Client SDK init, login / logout / token refresh
src/js/app.js	App bootstrap, route guards, UI rendering

10.2 Production API Configuration

In production, Vite bundles all JS/CSS into src/dist/. The Express backend serves this dist folder statically. All API calls from the frontend use relative paths (/api/...) so no CORS issues occur in production.

Socket.io URL is resolved dynamically:

- Development (localhost:5173): connects to http://localhost:5001
- Production (Render): connects to window.location.origin

11. Deployment Documentation

11.1 GitHub Repository

https://github.com/ShidoStack/smart-library

A single monorepo contains both backend/ and src/ (frontend). The .gitignore excludes node_modules, .env files, serviceAccountKey.json, and build output (dist/).

11.2 Render Unified Server Configuration

Setting	Value
Service Type	Web Service
Repository	smart-library (GitHub)
Branch	main
Language	Node
Region	Singapore
Build Command	npm run build (installs frontend + builds Vite dist + installs backend)
Start Command	npm start (node backend/src/server.js)
Plan	Free
Live URL	https://smart-library-hxsr.onrender.com

11.3 Render Environment Variables (Secrets)

Variable	Source
MONGODB_URI	MongoDB Atlas connection string with credentials
JWT_SECRET	Randomly generated strong secret
FIREBASE_SERVICE_ACCOUNT	Full JSON string of Firebase service account key
NODE_ENV	production
CLIENT_URL	https://smart-library-hxsr.onrender.com/

12. Testing and Verification

A comprehensive automated E2E test suite was created at backend/src/scripts/comprehensive_test.js covering all 35 test cases:

Area	Verification	Status
Backend startup	Express server starts, port listens	✅ Passed
MongoDB Atlas	Mongoose connection established	✅ Passed
Health Check API	GET /api/health returns 200 ok	✅ Passed
User Registration	POST /api/auth/register creates user	✅ Passed
User Login	POST /api/auth/login returns JWT	✅ Passed

Firebase Auth	Firebase Admin SDK initialised	✅ Passed
RBAC — Member access	Member denied admin-only routes (403)	✅ Passed
Books CRUD	Create / read / update / delete	✅ Passed
Books Search	GET /api/books?search= returns results	✅ Passed
Members CRUD	Create / read / update tested	✅ Passed
Borrow Book	POST /api/borrow decrements availableCopies	✅ Passed
Return Book	PUT /api/borrow/:id/return increments copies	✅ Passed
Reservations	Create and cancel reservation	✅ Passed
Fines	Fine listing and user-specific fines	✅ Passed
Reports	Borrowing / popular books / member activity	✅ Passed
Notifications	Create, list, mark-read, mark-all-read	✅ Passed
Socket.io	Real-time connection and events	✅ Passed
Frontend build (Vite)	npm run build produces dist/ assets	✅ Passed
Production deployment	Render web service live and healthy	✅ Passed

13. Security and Production Notes

- All passwords are hashed with bcryptjs (salt rounds = 12) before storage.
- Firebase Admin SDK verifies the authenticity of all Firebase ID Tokens server-side.
- SmartLibrary JWTs are signed with a server-side secret and expire after 7 days.
- Protected endpoints return 401 Unauthorized if no valid Bearer token is provided.
- Admin-only routes return 403 Forbidden when accessed by member-role users.
- .env and serviceAccountKey.json are excluded from GitHub via .gitignore.
- MONGODB_URI, JWT_SECRET, and Firebase credentials are stored only in Render environment variables — never in source code or screenshots.
- Helmet.js sets security headers (XSS protection, HSTS, frame denial) on all responses.

- CORS is restricted to the configured CLIENT_URL origin.
- Render free tier services may spin down after 15 minutes of inactivity; the first cold-start request may be slower.

14. End-to-End User Flow

Member Flow

- Step 1: Open the deployed frontend: https://smart-library-hxsr.onrender.com/login.html
- Step 2: Register a new account or log in with Email/Password or Google OAuth.
- Step 3: Firebase authenticates and returns a Firebase ID Token.
- Step 4: Frontend exchanges the ID Token via POST /api/auth/firebase and receives a SmartLibrary JWT.
- Step 5: JWT is stored in sessionStorage; all subsequent API calls include it as Bearer token.
- Step 6: Browse the book catalogue — search by title, author, genre, or ISBN.
- Step 7: Borrow an available book: POST /api/borrow.
- Step 8: Reserve an unavailable book: POST /api/reservations.
- Step 9: View borrow history, fines, and notifications from the dashboard.
- Step 10: Return a book: PUT /api/borrow/:id/return.

Administrator Flow

- Step 1: Log in with an admin-role account.
- Step 2: Add, update, or delete books from the catalogue.
- Step 3: Manage member accounts and memberships.
- Step 4: View all borrow records, reservations, and overdue fines.
- Step 5: Generate borrowing reports, popular books list, and member activity summaries.
- Step 6: Send notifications to members.

15. Troubleshooting

Issue	Checks
Failed to fetch / CORS error	Confirm backend is live, API URL is correct, CLIENT_URL env var matches frontend origin.

401 Unauthorized	Log in again; verify Authorization: Bearer <JWT> header is sent. JWT may have expired.
403 Forbidden	Verify the logged-in account has the required admin role in MongoDB.
0 books shown after login	Check browser DevTools → Application → sessionStorage for token; check Network tab for /api/books response.
MongoDB auth error	Check MONGODB_URI in .env; verify Atlas user credentials and Network Access whitelist.
Render build failure	Check Root Directory, Build Command, Start Command, and package.json scripts.
Socket.io not connecting	Ensure Socket.io URL is dynamic (window.location.origin in production).
Firebase Admin error	Verify FIREBASE_SERVICE_ACCOUNT env var on Render contains valid JSON.

16. Project Links

Resource	URL
Live Application	https://smart-library-hxsr.onrender.com
GitHub Repository	https://github.com/ShidoStack/smart-library
API Health Check	https://smart-library-hxsr.onrender.com/api/health
API Docs (Swagger)	https://smart-library-hxsr.onrender.com/api-docs

17. Conclusion

The SmartLibrary System delivers a complete, production-ready web-based library management workflow. It combines a Vanilla JS + Vite frontend with a Node.js / Express.js backend, MongoDB Atlas for persistent storage, Firebase Authentication for secure identity management, and Socket.io for real-time updates.

Role-based access control, JWT session management, automatic fine calculation, book reservation queuing, and real-time notifications ensure a reliable, feature-rich experience for both library members and administrators. The project has been built and

tested locally with a 35/35 passing E2E test suite and successfully deployed as a unified production server on Render.